Face Recognition

Face Recognition is a set of 2 tasks:

- **Face identification:** Given a face image that belongs to a person in a database, tell whose image it is
    - Has a database of K persons
    - Get an input image
    - Output: ID if the image is any of the K persons or “not recognized”
- **Face verification:** Given a face that might not belong to the database, verify whether it is from the person it is claimed to be in the database
    - Input: Image of a person, name / ID
    - Output: Whether the input image is that of the claimed person

Importance:

- The forefront module for any automatic face recognition system
- The first process in many Human Computer Interactive system
    - Expression recognition
    - Cognitive state / Emotional state recognition
    - Biometric security solution
- First step in many surveillance system
    - Tracking
    - Automatic Target recognition or generic object detection algorithm
    - Human behaviour analysis

Classical face recognition algorithms:

- **Holistic features: eigenfaces (PCA)**
    - Advantage: Easy to implement, no knowledge (such as facial features) required except face ID
    - Disadvantage: Algorithm sensitive to lighting, shadows and scale of the face in the image
- **Local based features: Local binary pattern (LBP), SIFT**
    - Advantage: Robust features in frontal face. Sensitive to nose and invariant to translations and rotations

Challenges:

- Classical approaches would not perform well in very large datasets of faces and limit the learning of rich and robust representations of faces
- The recent advent of affordable, powerful GPUs and the creation of huge face databases has drawn research focus primarily on the development of increasingly DNNs designed for all aspects of face recognition tasks, ranging from detection and preprocessing to feature representation and classification in verification and identification solutions

One-shot Learning

One-shot learning is a classification task that is used to form a model with only **1 example from each class**, which in turn will be used to make predictions about many unknown examples in the future

- Face recognition tasks provide examples of one-shot learning
- Specifically, in the case of face identification, a model or system may only have 1 or a few examples of a given person's face and must correctly identify the person from a new photographs with changes to expression, hairstyle, lighting, accessories and more
- In the case of face verification, a mode or system may only have 1 example of a person's face on record and must correctly verify new images / photos of that person, perhaps each day
- As such, face recognition is a common example of one-shot learning

Repurposing CNNs for One-shot learning

- Instead of treating the task as a classification problem, one-shot learning turns it into a **difference-evaluation problem**
- Learning a "similarity" function $d(img1, img2) = degree\ of\ difference\ between\ images$
- For face recognition, similarity function is used to check for every pair of the images
- If the images contain the same face, the neural network returns a value that is smaller than a specific threshold, and if they are not the same face, it will be higher than the threshold

Siamese Network

- Siamese neural network is not much different from other CNNs. It takes images as input and encodes their features into a set of numbers. The difference comes in the output processing
- During the training phase, classic CNNs tune their parameters so that they can associate each image to its proper class. The Siamese neural network, on the other hand, trains to be able to measure the distance $||f(x^{(1)} - f(x)^{(2)}||_2^2$ between the features in 2 input images
- Parameters of neural network define an encoding $f(x^{(1)})$. The goal is to learn parameters so that
    - If $x^{(i)}$, $x^{(j)}$ are the same person, $||f(x^{(i)} - f(x)^{(j)}||_2^2$ is small
    - If $x^{(i)}$, $x^{(j)}$ are the different person, $||f(x^{(i)} - f(x)^{(j)}||_2^2$ is large

Triplet Loss

- Basically, the triplet loss trains the neural network by giving it 3 images:
  - An **Anchor** image
  - A **Positive** image
  - A **Negative** image
- After the training, positive examples will be closer to the anchor while the negative examples will be farther from it
- To prevent deep model output 0 for all the encodings or learn the similar encoding for every images, we add a margin:

$$||f(A) - f(P)||^2 - ||f(A) - f(N)||^2 + margin \leq 0$$

- Margin defines how far away the dissimilarities should be in order to distinguish the 2 images better

Loss function formulation

- Given 3 images $A, P \; and \; N$:

$$L(A, P, N) = max(||f(A) - f(P)||^2 - ||f(A) - f(N)||^2 + \alpha, 0)$$

- Notice that, to train the model, we need some pairs of A and P for training, meaning pairs of pictures of the same person
- After having the trained model, we can apply in the one-shot learning problem in the face recognition, where maybe you have only single photo of the person we want to recognize

Choosing the Triplets A, P, N

- During training, if $A, P \; and \; N$ are chosen randomly, $d(A, P) + \alpha \leq d(A, N)$ is easily satisfied
- We have to choose triplets that are “hard” to train in order to increase the learning efficiency (choose the samples where the $d(A, P)$ close to $d(A, N)$

Triplet Mining

- Based on the definition of the loss, there are 3 categories of triplets:
  - Easy triplets: Triplets which have a loss of 0 because: $d(A, P) + margin < d(A, N)$
  - Hard triplets: Triplets where the negative is closer to the anchor than the positive: $d(A, N) < d(A, P)$
  - Semi-hard triplets: Triplets where the negative is not closer to the anchor than the positive, but which still have positive loss: $d(A, P) < d(A, N) < d(A, P) + margin$

- Each of these definitions depend on where the negative is, relatively to the anchor and positive
  ⇒ We can therefore extend these 3 categories to categories of negatives:
  Hard negatives, Semi-hard negatives, Easy negatives
- An important decision of a training with Triplet Loss is negative selection or **Triplet mining**. The strategy chosen will have a high impact on the training efficiency and final performance.
  - An obvious appreciation is that training with Easy Triplets should be avoided, since their resulting loss will be 0
  - **Offline Triplet Mining:** In this approach, we first generate the triplets manually and then fit the data to the network
  - **Online Triplet Mining:**
    - In this approach, we feed a batch of training data, generate triplets using examples in the batch and calculate the loss on it.
    - This approach allows us to randomize the triplets and increase the chance to find triplets with high losses → help faster training
    - For batch size of N, we can generate at most N^3 triplets

Embedding

The embedding can be used to compare faces in 3 ways:

- **Face Verification:**
  - This considers 2 faces and it decides whether they are similar or not
  - This can be done by computing the distance metric
- **Face Recognition:**
  - This is a classification problem for labelling a face with a name
  - The embedding vector can be used for training the final labels
- **Face Clustering:**
  - This groups similar faces together just like how photo applications cluster photos of the same person together
  - A clustering algorithm such as K-means is used to group faces

Types of Similarity Learning metrics

1. Cardinality based Similarity metrics

- These metrics leverage on the union and intersection of sets in comparison. The Jaccard similarity metric uses cardinality to determine the relation between 2 vectors

$$d(x^{(i)}, x^{(j)}) = \frac{|x^{(i)} \cap x^{(j)}|}{|x^{(i)} \cup x^{(j)}|}$$

- It is the size of the overlap between the 2 sets (what they have in common) divided by the size of the combined sets (everything they have without duplicates)
- A higher value means the sets are more similar

2. Orientation based Similarity metrics

- These metrics leverage the angle between 2 vectors in their respective vector spaces. An example is the Cosine similarity metric, it calculates similarity by measuring the cosine of angle between 2 vectors:

$$d(x^{(i)}, x^{(j)}) = \frac{\sum_{k=1}^{n} |x_k^{(i)} x_k^{(j)}|}{\sqrt{\sum_{k=1}^{n} x_k^{(i)^2}} \sqrt{\sum_{k=1}^{n} x_k^{(j)^2}}}$$

- It measures the similarity by calculating the cosine of the angle between 2 vectors in a multidimensional space
- If the Cosine similarity is close to 1, it means the vectors are pointing in roughly the same direction, indicating that the items we are comparing are similar

3. Distance based Similarity metrics

- These metrics are the Euclidean distance, Manhattan distance, Minkowski distance, and so on. The basic idea common to these metrics is that they leverage the average distance between elements of 2 vectors under experiment
    - Euclidean distance:

      $$d(x^{(i)}, x^{(j)}) = \sqrt{(x_1^{(i)} - x_1^{(j)})^2 + (x_2^{(i)} - x_2^{(j)})^2 + \ \dots \ + (x_n^{(i)} - x_n^{(j)})^2}$$

    - Manhattan distance:

      $$d(x^{(i)}, x^{(j)}) = |x_1^{(i)} - x_1^{(j)}| + |x_2^{(i)} - x_2^{(j)}| + \ \dots \ + |x_n^{(i)} - x_n^{(j)}|$$

    - Minkowski distance:

      $$d(x^{(i)}, x^{(j)}) = (\sum_{k=1}^{n} |x_k^{(i)} - x_k^{(j)}|^p)^{\frac{1}{p}}$$

- If we set $p = 2$, we get the Euclidean distance, if we set $p = 1$, we get the Manhattan distance. We can also set p to other infinite numbers

Face Verification and Binary Classification

- These is another way to learn neural network parameters beside Triplet loss is Binary classification problem
- To do that, we have to select a pair of neural networks, says the Siamese network. We simultaneously compute the embedding of image pairs, such as 128-dimensional embedding, or more, which are then fed into the logistic regression unit, and then move on to the next step to make predictions
- The final layer is a sigmoid layer:

  $Y' = w^{(i)} * Sigmoid(f(x^{(i)}) - f(x^{(j)})) + b$

  Where the subtraction is the Manhattan distance between $f(x^{(i)}) - f(x^{(j)})$
- Some other similarities can be Euclidean and chi-square similarity
- The neural network here is Siamese, means the top and bottom convolutions has the same parameters
- <u>A good deployment trick:</u>
  - Pre-compute all the images we use as a comparison to the vector $f(x^{(j)})$
  - When a new image that needs to be compared, get its vector $f(x^{(i)})$ then put it with all the pre computed vectors and pass it to the Sigmoid function

<u>In short:</u>
There are 2 different approaches for face recognition:

- Minimize the loss of triplets by training on the triplet A, P, N
- Use a training set containing 2 images with only targets of 1 or 0 and trained by logistic regression